1.**Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.**

ALGORITHM:

1. Read n strings and store them in an array.
2. Check each string from left to right by comparing it with its reverse.
3. Return the first palindromic string; if none exists, return "".

PYTHON CODE:

```
def firstPalindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""
n = int(input("Enter number of words: "))
words = []
for i in range(n):
    words.append(input("Enter word: "))
print("First palindrome:", firstPalindrome(words))
```

OUTPUT:

```
========================================
Enter number of words: 3
Enter word: son
Enter word: racecar
Enter word: honey
First palindrome: racecar
```

COMPLEXITY:

TIME COMPLEXITY: **O(n × m)**

SPACE COMPLEXITY: **O(m)**

2.You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

ALGORITHM:

1. Convert both arrays into sets for fast element searching.
2. Count elements of nums1 present in nums2, and elements of nums2 present in nums1.
3. Return the two counts as [answer1, answer2].

PYTHON CODE:

```
nums1 = list(map(int, input("Enter nums1: ").split()))
nums2 = list(map(int, input("Enter nums2: ").split()))

set1 = set(nums1)
set2 = set(nums2)

answer1 = sum(x in set2 for x in nums1)
answer2 = sum(x in set1 for x in nums2)

print([answer1, answer2])
```

OUTPUT:

```
Enter nums1: 2 3 2
Enter nums2:  1 2
[2, 1]
>>>
```

COMPLEXITY ANALYSIS:

1. **Time:** O(n + m)
2. **Space:** O(n + m)

3.You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array.

ALGORITHM:

1. Start each subarray from every index i and maintain a set of distinct values.
2. Extend the subarray to index j, add the new element to the set, and calculate len(set)^2.
3. Add all these squared distinct counts and return the total.

PYTHON CODE:

```
nums = list(map(int, input("Enter nums: ").split()))

ans = 0
n = len(nums)

for i in range(n):
    s = set()
    for j in range(i, n):
        s.add(nums[j])
        ans += len(s) ** 2

print(ans)
```

OUTPUT:

```
Enter nums: 1 2 1
15
```

COMPLEXITY ANALYSIS:

1. **Time:** $O(n^2)$
2. **Space:** O(n)

1.Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by k.

ALGORITHM:

1. Read the array and k, then check every pair (i, j) where i < j.
2. If nums[i] == nums[j] and (i × j) % k == 0, increment the count.
3. Return the total number of valid pairs.

PYTHON CODE:

```
nums = list(map(int, input("Enter nums: ").split()))
k = int(input("Enter k: "))

count = 0
n = len(nums)

for i in range(n):
    for j in range(i + 1, n):
        if nums[i] == nums[j] and (i * j) % k == 0:
            count += 1

print(count)
```

OUTPUT:

```
Enter nums: 3 1 2 2 2 1 3
Enter k: 2
4
```

COMPLEXITY ANALYSIS:

1. **Time:** $O(n^2)$
2. **Space:** $O(1)$

2.Write a program FOR THE BELOW TEST CASES with least time complexity

ALGORITHM:

1. Read the array and initialize max with the first element.
2. Compare each remaining element with max; if it is larger, update max.
3. Print max as the largest element.

PYTHON CODE:

```
nums = list(map(int, input("Enter elements: ").split()))

max_num = nums[0]

for x in nums[1:]:
    if x > max_num:
        max_num = x

print(max_num)
```

OUTPUT:

```
Enter elements: 1 2 3 4 5
5
>
```

COMPLEXITY:

Time Complexity: O(n)
Space Complexity: O(1)

3.You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

ALGORITHM:

1. Read the list of numbers and check whether it is empty.
2. If not empty, sort the list using sort().
3. Return the last element of the sorted list as the maximum.

PYTHON CODE:

```
nums = list(map(int, input("Enter elements: ").split()))

if not nums:
    print("List is empty")
else:
    nums.sort()          # Efficient sorting
    print("Maximum:", nums[-1])
```

OUTPUT:

```
== RESTART: C:/Users/aswin/
Enter elements: 5
Maximum: 5
```

COMPLEXITY:

Time Complexity: O(n log n)
Space Complexity: O(n)

4.Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

ALGORITHM:

- Read the list of n numbers.
- Use a dictionary/set to remove duplicate elements.
- Store and print only the unique elements.

PYTHON CODE:

```
nums = list(map(int, input("Enter elements: ").split()))

unique = list(dict.fromkeys(nums))

print(unique)
```

OUTPUT:

```
Enter elements: 3 7 3 5 2 5 9 2
[3, 7, 5, 2, 9]
```

COMPLEXITY:

- **Time Complexity:** O(n)
- **Space Complexity: O(n)**

4. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

ALGORITHM:

- Compare adjacent elements in the array.
- Swap them if they are in the wrong order.
- Repeat the passes until the array is sorted.

PYTHON CODE:

```
nums = list(map(int, input("Enter elements: ").split()))

n = len(nums)

for i in range(n):
    for j in range(0, n - i - 1):
        if nums[j] > nums[j + 1]:
            nums[j], nums[j + 1] = nums[j + 1], nums[j]

print("Sorted array:", nums)
```

OUTPUT:

```
== RESTART: C:/Users/aswin/AppDa
Enter elements: 5 2 8 1 4
Sorted array: [1, 2, 4, 5, 8]
```

COMPLEXITY:

- **Best Case:** O(n)
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Space Complexity: O(1)

6.Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

ALGORITHM:

- Set low = 0 and high = n - 1.
- Find the middle element. If it equals the key, return its position; if the key is greater, search the right half, otherwise search the left half.
- Repeat until the key is found or low > high. If not found, display **Element not found**.

PYTHON CODE:

```
arr = list(map(int, input("Enter sorted array: ").split()))
key = int(input("Enter key: "))

low, high = 0, len(arr) - 1
found = -1

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        found = mid
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1

if found != -1:
    print(f"Element {key} is found at position {found + 1}")
else:
    print(f"Element {key} is not found")
```

OUTPUT:

```
== RESTART: C:/Users/aswin/AppData/Local/Programs/P
Enter sorted array: -9 3 4 6 8 9 10 30
10
Enter key: Element 10 is found at position 7
```

COMPLEXITY:

Time Complexity: O(log n)
Space Complexity: O(1)

7.Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

ALGORITHM:

- Divide the array into two halves recursively.
- Sort both halves and merge them in ascending order.
- Continue until the complete array is sorted.

PYTHON CODE:

```
nums = list(map(int, input("Enter elements: ").split()))

def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

print(merge_sort(nums))
```

OUTPUT:

```
== RESTART: C:/Users/aswin/AppData
Enter elements: 5 2 8 1 4 3
[1, 2, 3, 4, 5, 8]
```

COMPLEXITY:

- **Time:** O(n log n)
- **Space:** O(n)

8.Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

ALGORITHM:

- Store the number of ways to reach each cell using dynamic programming.
- For each step, move the ball in four directions; if a move goes outside the grid, add it to the answer.
- Repeat for exactly N steps and print the total number of ways.

PYTHON CODE:

```
m, n, N, i, j = map(int, input("Enter m n N i j: ").split())

dp = [[0] * n for _ in range(m)]
dp[i][j] = 1

ans = 0

for _ in range(N):
    new = [[0] * n for _ in range(m)]

    for r in range(m):
        for c in range(n):
            if dp[r][c]:
                for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
                    nr, nc = r + dr, c + dc

                    if 0 <= nr < m and 0 <= nc < n:
                        new[nr][nc] += dp[r][c]
                    else:
                        ans += dp[r][c]

    dp = new

print(ans)
```

OUTPUT:

```
== RESTART: C:/Users/aswin/AppData/
Enter m n N i j: 2 2 2 0 0
6
```

COMPLEXITY:

- **Time:** O(N × m × n)
- **Space:** O(m × n)

9.You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first

house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

ALGORITHM:

- Since the houses form a circle, consider two cases: **exclude the first house** or **exclude the last house**.
- For each case, use dynamic programming to find the maximum money from non-adjacent houses.

Return the maximum of the two cases.

PYTHON CODE:

```
nums = list(map(int, input("Enter house money: ").split()))

def rob(a):
    prev = curr = 0

    for money in a:
        prev, curr = curr, max(curr, prev + money)

    return curr

if not nums:
    print(0)
elif len(nums) == 1:
    print(nums[0])
else:
    # Either skip the first house or skip the last house
    answer = max(rob(nums[:-1]), rob(nums[1:]))
    print(answer)
```

OUTPUT:

```
-- RESTART: C:/Users/aswin/
Enter house money: 2 3 2
3
```

COMPLEXITY:

- **Time:** O(n)
- **Space:** O(1)

10.You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

ALGORITHM:

- Initialize two variables to represent the number of ways for the previous steps.
- For each step, calculate the new number of ways as the sum of the previous two.
- Print the number of ways for n steps.

PYTHON CODE:

```
n = int(input("Enter n: "))

a, b = 1, 1

for _ in range(n):
    a, b = b, a + b

print(b)
```

OUTPUT:

```
Enter n: 4
8
```

COMPLEXITY:

- **Time:** O(n)
- **Space:** O(1)